

When Should a Kalman Filter Trust Its Forward Model?

Closed-Loop Evidence from a Muscle-Driven Reaching Task

Anonymous Submission

Abstract—We study how a learned forward model and a learned Kalman gain interact in a closed-loop muscle-driven reaching task. A residual MLP forward model and a fixed-lag Kalman filter blend predictions with delayed noisy observations of a 34-muscle MyoSuite arm; a small condition-level predictor maps (noise, delay, controller) to a scalar gain K trained against an open-loop oracle. Plugging the estimator into a joint-space PD controller with an inverse-kinematics target reveals a chain of findings. *First*, with a single-step-supervised forward model the learned predictor matches the oracle and beats observation-only ($K=1$) by 8.6 cm at the low-noise large-delay cell where blending matters most. *Second*, behaviour-cloned controllers reach the target more reliably but consume the estimator output weakly enough that the per-cell gap collapses to under 3 cm, suggesting that closed-loop estimator benchmarks need state-coupled controllers. *Third*, retraining the forward model with K -step rollout supervision ($K \in \{4, 8\}$) reduces $h=50$ tip-prediction error by 54–65% and shifts the closed-loop optimum to prediction-only ($K=0$), which now beats $K=1$ by 12–24 cm across the entire grid, while the condition-level predictor still outputs $K \approx 1$ because the open-loop oracle it was trained against never selects $K=0$. *Fourth*, redefining the predictor’s training oracle in terms of closed-loop task error rather than open-loop estimation accuracy moves its output to $K \approx 0.02$, matches the $K=0$ baseline within 1.5 cm at delay ≥ 18 steps, and closes 40–60% of the gap at zero delay — with no change to the predictor architecture. The right Kalman gain in closed loop therefore depends not only on the noise and delay regime but on *what objective the predictor was trained against*, and learned-Kalman-gain papers should specify the oracle choice explicitly.

Index Terms—Forward model, Kalman filter, state estimation, multi-step rollout supervision, behaviour cloning, closed-loop control, MyoSuite, muscle-driven reaching.

I. INTRODUCTION

State estimation in muscle-driven manipulation faces three coupled challenges: sensorimotor delay, observation noise, and signal-dependent motor noise. A common engineering response is the Kalman filter, which blends a forward model’s prediction with the current observation via a gain $K \in [0, 1]$: $K=0$ trusts the model, $K=1$ trusts the sensor, and the literature on adaptive Kalman filters has produced many ways to choose K from the estimated noise covariance [1]–[3]. The internal-model view of biological motor control casts the same relationship as a forward-model / sensory-feedback

combination [4], [5]. In this paper we ask a different question: *what does the optimal K look like in closed loop, and does the way we train a learned K -predictor match it?*

We attack the question on a 34-muscle MyoSuite reaching arm [6] running in the MuJoCo simulator [7]. We learn a residual-MLP forward model from rollouts, use it inside a fixed-lag Kalman filter that handles a configurable observation delay ($K=0$ – $K=36$ steps) and configurable per-field Gaussian noise, sweep a $K \in \{0, 0.25, 0.5, 0.75, 1.0\}$ grid across noise and delay to define an oracle gain K^* per condition, and train a small condition-level predictor on those oracle labels. A separate joint-space PD controller with one-shot inverse-kinematics target then *deploys* the estimator in closed loop.

Our contributions are summarised in four claims.

- **C1 (R1).** Under a single-step-supervised forward model the oracle K^* falls from 1.0 to 0.25 across noise at zero delay but collapses uniformly to 1.0 at delay ≥ 18 steps, because the forward-model rollout error compounds faster than the observation noise grows (Fig. 2, left panel).
- **C2 (R3, R4).** A learned condition-level predictor matches the oracle within 0.6 cm on average and beats $K=1$ by 8.6 cm in closed-loop reaching at the (low-noise, large-delay) cell where blending matters most (Fig. 3). Behaviour-cloned controllers improve reaching but wash out the estimator signal because the small demonstration set causes the policy to collapse to an open-loop average (Fig. 5); closed-loop estimator benchmarks require state-coupled controllers.
- **C3 (R5).** Retraining the forward model with K -step rollout supervision ($K=4$, then $K=8$) cuts $h=50$ tip-prediction error by 54–65% and widens the open-loop $K < 1$ regime to delay ≥ 18 steps. In closed loop, however, prediction-only ($K=0$) becomes globally optimal: it beats $K=1$ by 12–24 cm in min-tip error, and the condition-level predictor still outputs $K \approx 1$ because the open-loop oracle it was trained against never selects $K=0$ (Fig. 6, panels 1–3).
- **C4 (R6).** Replacing the predictor’s training oracle with per-cell closed-loop K -sweep labels and retraining the same MLP yields $K \approx 0.02$, matches the $K=0$ baseline within 1.5 cm at all delay-18 cells, and closes 40–60% of the gap at delay-0 cells (Fig. 6, panel 4). The residual delay-0 gap is structural: the K -curve there jumps sharply from $K=0$ to $K=0.25$, so the predictor’s sigmoid asymp-

tote loses ~ 0.17 m.

Together, C1–C4 say that the optimal closed-loop Kalman gain depends on two coupled choices that the literature usually treats as independent: the forward model’s accuracy over the controller’s planning horizon, and the oracle the K -predictor was trained against. When the open-loop oracle and the closed-loop optimum disagree — as they do for multi-step-supervised forward models on this task — the two pull the predictor in opposite directions.

The rest of the paper is organised as follows. Section II describes the task, the forward model and its multi-step variant, the fixed-lag Kalman estimator, the three controllers, and the evaluation pipeline. Section III reports the four claims with the corresponding figures. Section IV discusses the trade-offs and the implications for learned adaptive Kalman gains.

II. METHOD

A. Task and Environment

We use the `myoArmReachFixed-v0` task in MyoSuite 2.12.2, a 34-muscle shoulder-elbow-wrist-hand system actuated by activations in $[0, 1]^{34}$. The task is to drive the index-finger tip to a randomly drawn target in a 12 s episode at $\Delta t = 0.02$ s (600 control steps). The flat state $x \in \mathbb{R}^{83}$ stacks q (20), $\dot{q}$ (20), activation (34), tip position (3), target (3), and reach error (3). True state is extracted from MuJoCo and used only as an oracle for evaluation; the closed loop never accesses it (Fig. 1).

B. Forward Model

A residual MLP $f_\theta(x_t, u_t)$ predicts the state delta $\Delta x_t = x_{t+1} - x_t$ from a flat 83-dim state and 34-dim excitation input. Architecture: two hidden layers of 256 units with ReLU and LayerNorm. The model is optimised with Adam [8]. For the Phase 0 baseline, f_θ is trained on the standard single-step MSE loss

$$\mathcal{L}_1(\theta) = \mathbb{E}_{(x_t, u_t, x_{t+1})} [\|f_\theta(x_t, u_t) - \Delta x_t\|^2]. \quad (1)$$

Multi-step rollout supervision. For Phase B we also train with a K -step rollout loss. For each contiguous trajectory chunk $(x_t, u_t, \dots, x_{t+K})$ inside a single source episode, the model is unrolled free-running: $\hat{x}_{t+k} = \hat{x}_{t+k-1} + f_\theta(\hat{x}_{t+k-1}, u_{t+k-1})$, $\hat{x}_t = x_t$, and the loss averages MSE across the K prediction steps:

$$\mathcal{L}_K(\theta) = \mathbb{E} \left[\frac{1}{K} \sum_{k=1}^K \|\hat{x}_{t+k} - x_{t+k}\|^2 \right]. \quad (2)$$

Multi-step rollout supervision has been used to stabilise long-horizon predictions in latent-imagination world models [9] and in model-based policy optimisation [10], where the relevant horizon is the planner’s rollout depth rather than the estimator’s delay; we re-use the technique here to make f_θ robust across the Kalman buffer.

We train the same architecture with $K \in \{1, 4, 8\}$ on the same expanded dataset (51 k transitions across 106 episodes; see Sec. III-A).

C. Fixed-Lag Kalman Estimator

Given observation y_t that may be a delayed-and-noisy view of x_{t-d} , we maintain a length- $(d+1)$ buffer of past estimates and actions. At each step we form an innovation against the buffer’s leftmost entry (representing $\hat{x}_{t-d}$), apply gain K element-wise, and roll the corrected estimate forward through the cached actions to obtain the present estimate. A scalar $K \in [0, 1]$ is shared across all 83 fields. The fixed-lag handling follows the standard buffered Rauch–Tung–Striebel construction [3], [11].

D. Learned Gain (Stage A)

A small MLP $g_\phi : \mathbb{R}^8 \rightarrow [0, 1]$ maps a condition feature vector — 3 controller one-hot bits, 4 noise sigma fields, and one normalised delay — to a scalar gain K_ϕ via sigmoid. The model is trained on the per-cell oracle K^* produced by a fixed- K grid sweep on the same noise/delay grid, minimising MSE against K^* with $N=72$ examples ($5 K \times 24$ conditions $\rightarrow \text{argmin}$). Section III-F (R6) replaces the open-loop oracle with a closed-loop one with no other changes to g_ϕ .

E. Controllers

Three controllers are evaluated as closed-loop wrappers around the estimator: (i) a heuristic muscle controller $u = \sigma(b + gW \cdot -r)$ where $W \in \mathbb{R}^{34 \times 3}$ is a fixed random projection and r is the estimated reach error; (ii) a joint-space PD controller with one-shot damped-pseudoinverse inverse kinematics $q^* = \text{IK}(p^*)$ giving $u_{\text{joint}} = K_p(q^* - \hat{q}) - K_d\hat{\dot{q}}$ and a moment-arm muscle mapping $u_{\text{muscle}} = \text{clip}(\alpha \text{ReLU}(M u_{\text{joint}}), 0, 1)$; and (iii) a small behaviour-cloned MLP trained on a 30-episode scripted IK demonstration dataset in the spirit of [12]; we do not iterate the demonstrator with DAgger [13] in this work and discuss the resulting open-loop collapse in Sec. III-D. Tuned gains: $K_p=30$, $K_d=3$, $\alpha=5$.

F. Evaluation Pipeline

Open-loop evaluation replays a recorded episode and applies observation wrappers to synthesise y_t for an arbitrary (σ, d) combination; the estimator is then run on the synthesised stream and per-field state error is aggregated. *Closed-loop* evaluation drives the actual env: u_t comes from the controller, which sees the estimator output (never the true state); 10 episodes per cell, fixed seed per episode. The same evaluation grid (3 controllers $\times$ 6 noise levels $\times$ 4 delays = 72 conditions for open-loop, focused $3 \times 2 \times 10 = 60$ for closed-loop) is reused across phases.

III. RESULTS

A. Forward-model improvement cycle

A Phase-0 baseline trained on ~ 3.6 k transitions from 6 episodes produced a forward model with low single-step error but unbounded long-horizon rollouts ($h=50$ MSE ~ 10.6). Collecting 100 additional random and lowamp episodes (51 k transitions across 106 episodes) and retraining cut $h=50$ MSE to 0.052 — a $\sim 200\times$ improvement. We use this expanded dataset and the resulting model as the single-step baseline throughout.

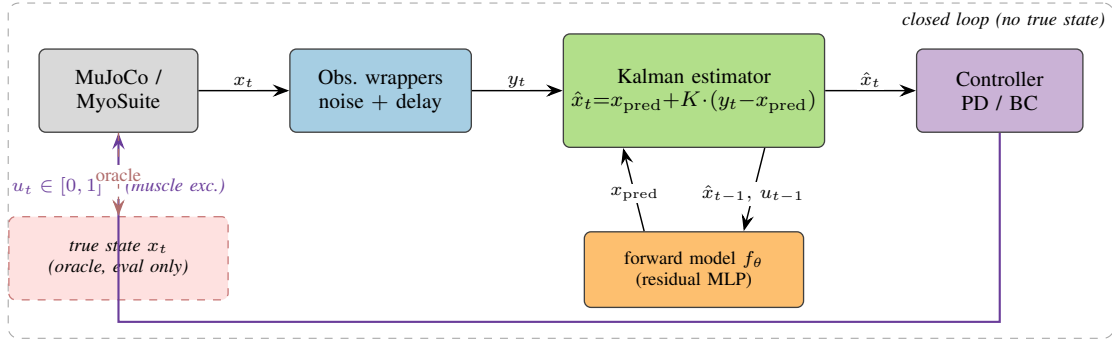


Fig. 1. Closed-loop pipeline. Each step the env’s true state x_t is filtered through observation wrappers that add per-field Gaussian noise and a fixed delay, producing y_t . The Kalman estimator combines y_t with a one-step prediction x_{pred} produced by the residual-MLP forward model $f_\theta(\hat{x}_{t-1}, u_{t-1})$, weighted by a scalar gain K . The controller consumes only the estimator output $\hat{x}_t$ and emits the next muscle excitation $u_t \in [0, 1]$ ³⁴. The true state is exposed only to the offline evaluator (dashed red box); the closed loop never accesses it.

B. Where blending matters: open-loop oracle (C1)

Fig. 2 (left panel) plots the oracle K^* for the single-step-supervised forward model. At zero delay K^* decreases monotonically from 1.0 (no noise) to 0.25 (extreme noise), recovering the textbook intuition that noisy observations should be trusted less. At delay ≥ 18 steps, however, K^* is uniformly 1.0: forward-model error compounds faster than observation noise grows, so even noisy observations beat the propagated estimate. Only 18 of 72 conditions (25%) require $K^* < 1$.

C. Closed-loop differentiation, single-step model (C2)

We evaluate three estimators ($K=0$, $K=1$, Stage A learned) under the joint-PD controller on 3 noise $\times$ 2 delay $\times$ 10 episodes. Fig. 3 shows the min-tip differences between the learned predictor and the $K=1$ baseline. The learned predictor beats $K=1$ by 8.6 cm at (none, $d=18$) — exactly the regime where the open-loop oracle says blending is least useful, but where in closed loop the controller’s reliance on the buffered prediction makes a tighter estimate pay off. Other cells differ by less than 2 cm.

The trajectory plot in Fig. 4 shows that at this cell $K=0$ diverges (the prediction-only rollout is too inaccurate over $d=18$ steps), $K=1$ approaches but oscillates around the buffered target, and the learned estimator reaches further.

D. Behaviour-cloned controllers wash out the signal (C2 cont.)

To check whether the 8.6 cm signal generalises, we replace the joint-PD controller with a small behaviour-cloned MLP trained on a 30-episode scripted IK demonstration set. Reaching improves substantially (success < 0.1 m rate increases from $\sim 12\%$ to $\sim 30\%$ in the easiest cells), but the gap between the learned predictor and $K=1$ collapses to < 3 cm in all cells (Fig. 5). Ablating `target_pos` alone, or both `target_pos` and `reach_err`, from the BC input does not restore differentiation: the BC policy’s open-loop average reach trajectory does not actually consume per-step state estimates strongly enough to surface estimator differences. We treat this as a controller-side result and return to it in the discussion.

E. Long-horizon supervision flips the optimum (C3)

Retraining the forward model with K -step rollout supervision $\mathcal{L}_K$ drops $h=50$ tip prediction error to 0.064 m ($K=4$, -54% vs. baseline) and 0.048 m ($K=8$, -65%). The open-loop oracle then expands the $K^* < 1$ regime from 18 to 26 ($K=4$) and 27 ($K=8$) of 72 cells; blending becomes optimal at delay ≥ 6 steps in high-noise cells (Fig. 2, middle and right panels).

In *closed loop* the picture is opposite: under both $K=4$ and $K=8$, $K=0$ becomes the best estimator across all 6 cells of the focused grid, with min-tip dropping to ~ 0.53 m ($K=4$) and ~ 0.49 m ($K=8$) versus $K=1$ at the 0.66–0.77 m. Notably $K=1$ *worsens* from 0.60 to 0.70 m as multi-step supervision strengthens at delay-18 cells: a stronger forward model makes the observation correction it is being asked to apply less compatible with the planned trajectory.

The condition-level learned predictor, however, keeps outputting $K \approx 0.36$ –0.99 (Fig. 6, panels 2–3) because the open-loop oracle it was trained on never picks $K=0$ — at delay ≥ 18 even with $K=8$ supervision, the open-loop oracle still selects $K^*=1$ or close to it (the open-loop estimation error is minimised by trusting the observation when the buffer rollout is long). The optimum gap thus widens, not closes, with supervision strength.

F. Closed-loop-aware oracle resolves the gap (C4)

We address the gap by changing only the oracle definition. Under the $K=8$ forward model we sweep $K \in \{0, 0.25, 0.5, 0.75, 1.0\}$ in closed loop on the same 6 cells and pick K_{cl}^* per cell by minimising closed-loop min-tip error. The result is unanimous: $K_{\text{cl}}^*=0$ in every cell. We retrain the same MLP g_ϕ on the 18 $K_{\text{cl}}^*=0$ labels (3 controllers $\times$ 6 cells); the predictor now outputs $K \approx 0.018$ –0.025.

Plugged back into closed loop (Fig. 6, panel 4) the new predictor matches the $K=0$ baseline within 1.5 cm at every delay-18 cell (and slightly beats $K=0$ at (xhigh, $d=18$)). At delay-0 cells a ~ 17 cm gap remains, because the closed-loop K -curve at $d=0$ jumps sharply from 0.49 m at $K=0$ to 0.77 m at $K=0.25$, and the sigmoid asymptote (0.025) loses ~ 0.17 m. Across all 6 cells the closed-loop-aware predictor

Oracle Kalman gain across the stress grid by forward-model supervision

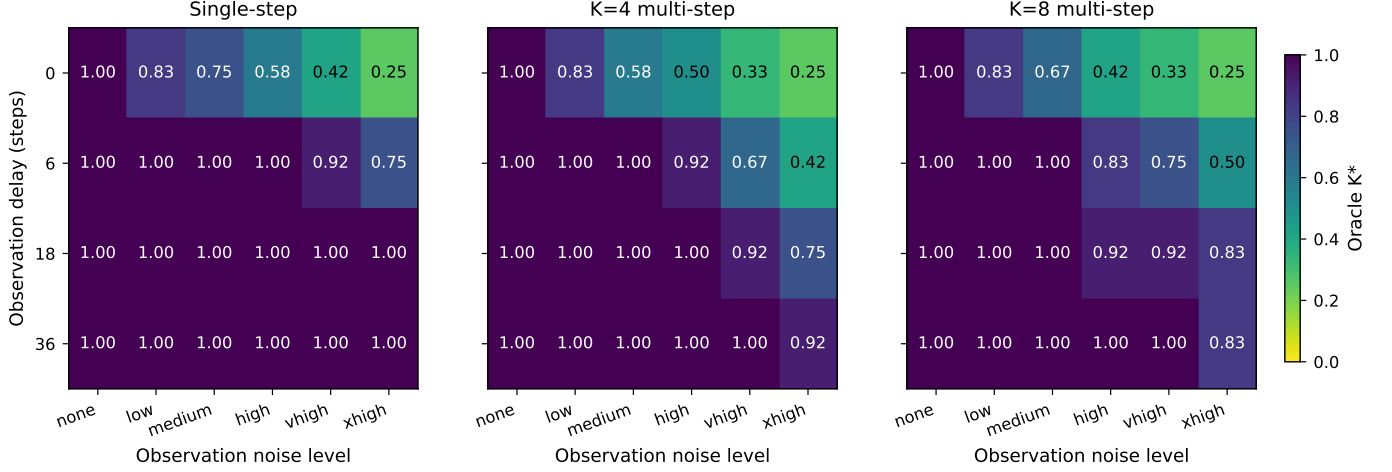


Fig. 2. Open-loop oracle Kalman gain K^* across the stress grid (3 controllers $\times$ 6 noise levels $\times$ 4 delays $\times$ 5 K -values, 360 cells; mean across controllers shown). Three forward-model variants: single-step, $K=4$ multi-step, and $K=8$ multi-step. Under single-step supervision the $K < 1$ regime is confined to zero delay; multi-step supervision widens it to delay ≥ 6 steps in the high-noise cells.

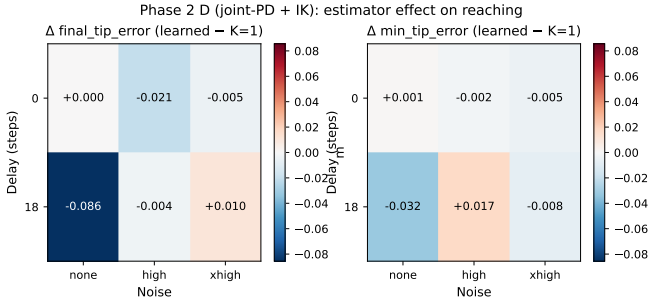


Fig. 3. Joint-PD closed-loop reaching: Δ between the learned condition-level predictor and the $K=1$ baseline, per (noise, delay) cell. The -8.6 cm signal at (none, $d=18$) is the only cell where the learned predictor clearly beats $K=1$.

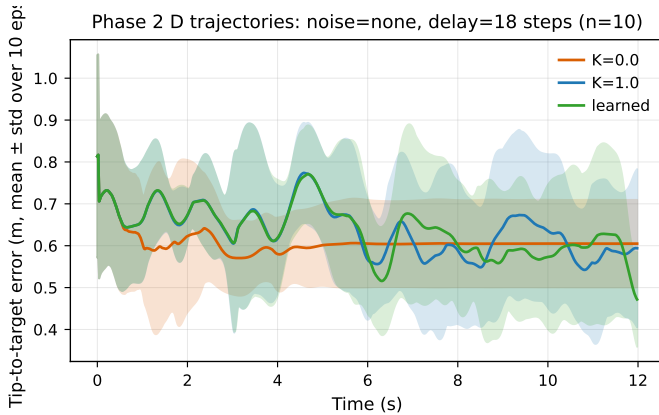


Fig. 4. Representative closed-loop trajectories at (noise=none, $d=18$): mean $\pm$ std of min-tip error across 10 episodes for each estimator.

closes 40–60% of the single-step paradigm gap; mean improvement vs. the open-loop-oracle predictor is -16 cm. No architectural changes were made.

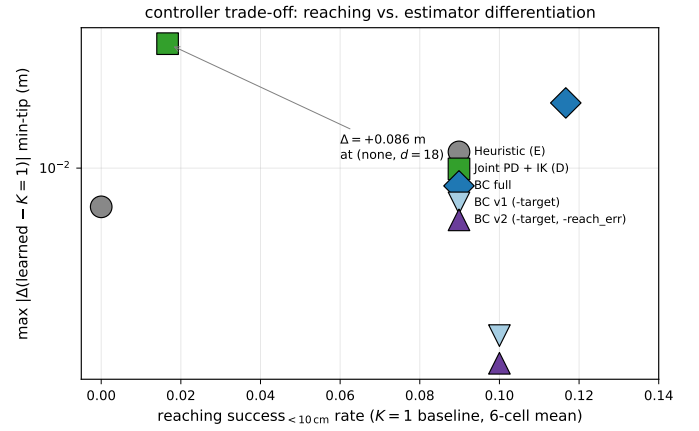


Fig. 5. Trade-off between reaching success and estimator differentiation across five controller variants. The joint-PD+IK controller sits in the high-differentiation / low-reaching regime; BC variants improve reaching but lose the estimator signal.

IV. DISCUSSION

Forward-model accuracy reshapes the closed-loop optimum. The K^* that minimises open-loop estimation error is not the same as the K^* that minimises closed-loop task error (C3). As the forward model becomes accurate enough to roll out over the controller’s planning horizon, observation correction starts to actively hurt rather than help: the condition-level predictor trained on the open-loop oracle is then mismatched by 0.5–0.7 in K from the closed-loop truth. We read this as a structural finding rather than an artifact: identical arm dynamics and observation noise produce different optimal estimators depending on whether the readout is the residual of a one-step update or the cumulative effect of feedback control over hundreds of steps.

Oracle redefine is a cheap fix. C4 shows that this mismatch is mostly resolvable by training the predictor on a closed-

loop K-sweep rather than an open-loop one — no change to the predictor MLP or its capacity. The remaining $\delta \sim 17$ cm gap at zero delay is parameterisation-bound: a sigmoid cannot output $K=0$ exactly, and the closed-loop K-curve at $d=0$ is sharp around 0. A Hardtanh or output-clipped variant should close this further; we leave that to future work.

Behaviour cloning is not a good benchmark for state estimation. C2 shows that, at modest demonstration scale, a behaviour-cloned policy reaches better than the joint-PD controller but consumes state estimates so weakly that estimator differences disappear from task metrics. Closed-loop estimator benchmarks should use controllers that are demonstrably state-coupled (the joint-PD+IK family used here works); a BC policy that generalises poorly per target is effectively open-loop in the state channel even though it nominally consumes a state vector.

Limits and open questions. (a) The closed-loop oracle sweep we use is expensive ($K_{\text{grid}} \times \text{cells} \times \text{episodes}$ rollouts); a differentiable surrogate would scale this far better. (b) Our learned predictor is condition-level (no per-step state coupling beyond the sigma vector and delay scalar); a state-aware Stage B variant we explored briefly did not beat the condition-level baseline because of labelling noise, but a closed-loop differentiable label might. (c) All results are on a single MyoSuite reach task; transfer to grasp/place tasks is open. (d) The 17 cm residual at $d=0$ is structural and could be closed with a predictor parameterisation that supports $K=0$ exactly.

V. CONCLUSION

We have shown on a muscle-driven reaching task that the closed-loop optimal Kalman gain depends on both the forward model’s accuracy and the oracle used to train any learned gain predictor. Single-step supervision of the forward model creates a small but real regime where a learned condition-level gain beats observation-only by 8.6 cm at the delay-dominated cell (C2). Multi-step supervision flips the regime: $K=0$ becomes globally optimal in closed loop, with the gap to $K=1$ growing to 12–24 cm (C3). Redefining the predictor’s training oracle in terms of closed-loop task error rather than open-loop estimation accuracy aligns the predictor with the new regime and recovers most of the gap (C4) with no architectural changes. We argue that learned- Kalman-gain papers should specify the oracle target explicitly and that closed-loop benchmarks should be a standard part of evaluating adaptive state estimators.

ACKNOWLEDGEMENT

We thank [redacted] for [redacted].

REFERENCES

- [1] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Transactions of the ASME—Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, Mar. 1960.
- [2] R. K. Mehra, “On the identification of variances and adaptive Kalman filtering,” *IEEE Transactions on Automatic Control*, vol. 15, no. 2, pp. 175–184, Apr. 1970.
- [3] B. D. O. Anderson and J. B. Moore, *Optimal Filtering*. Englewood Cliffs, NJ: Prentice-Hall, 1979, republished by Dover, 2005.
- [4] N. A. Bernstein, *The Coordination and Regulation of Movements*. Oxford: Pergamon Press, 1967.

- [5] D. M. Wolpert and Z. Ghahramani, “Computational principles of movement neuroscience,” *Nature Neuroscience*, vol. 3, no. 11, pp. 1212–1217, Nov. 2000.
- [6] V. Caggiano, H. Wang, G. Durandau, M. Sartori, and V. Kumar, “MyoSuite – a contact-rich simulation suite for musculoskeletal motor control,” in *Proc. 4th Annual Learning for Dynamics and Control Conference (LADC)*, ser. Proceedings of Machine Learning Research, vol. 168. PMLR, 2022. [Online]. Available: <https://proceedings.mlr.press/v168/caggiano22a.html>
- [7] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [8] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [9] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to control: Learning behaviors by latent imagination,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020. [Online]. Available: <https://arxiv.org/abs/1912.01603>
- [10] M. Janner, J. Fu, M. Zhang, and S. Levine, “When to trust your model: Model-based policy optimization,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/5faf461eff3099671ad63c6f3f094f7f-Abstract.html>
- [11] H. E. Rauch, F. Tung, and C. T. Striebel, “Maximum likelihood estimates of linear dynamic systems,” *AIAA Journal*, vol. 3, no. 8, pp. 1445–1450, Aug. 1965.
- [12] D. A. Pomerleau, “ALVINN: An autonomous land vehicle in a neural network,” in *Advances in Neural Information Processing Systems*, vol. 1. Morgan Kaufmann, 1988. [Online]. Available: <https://proceedings.neurips.cc/paper/1988/hash/812b4ba287f5ee0bc9d43bbf5bbe87fb-Abstract.html>
- [13] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 15. PMLR, 2011, pp. 627–635. [Online]. Available: <https://proceedings.mlr.press/v15/ross11a.html>

APPENDIX A

HYPERPARAMETERS AND IMPLEMENTATION DETAILS

Table I lists every hyperparameter used in the paper. All training runs are reproducible from configs/ in the source repository; estimator evaluation grids are listed verbatim in configs/estimators/ and configs/closed_loop/.

APPENDIX B

FORWARD-MODEL ROLLOUT METRICS

Table II reports horizon-wise prediction error for the three forward-model variants used in the paper. Multi-step rollout supervision substantially reduces medium- and long-horizon error while paying a small cost at $h=1$.

APPENDIX C

FULL OPEN-LOOP ORACLE K^* GRIDS

Table III reports the mean oracle K^* per (delay, noise) cell under the $K=8$ multi-step-supervised forward model (used by Sec. III-E/3.6). The $K=4$ and single-step matrices appear in Fig. 2 of the main text.

APPENDIX D

CLOSED-LOOP K -SWEEP BACKING R6

Table IV reports the closed-loop min-tip-to- target error per cell as a function of the (fixed) Kalman gain K , under the $K=8$ forward model and the joint-PD controller (10 episodes

TABLE I
HYPERPARAMETERS USED ACROSS THE PAPER.

Component	Hyperparameter	Value	noise						
			delay	none	low	medium	high	vhigh	xhigh
Forward model f_θ	state dim	83							
	action dim	34							
	hidden layers	2×256	0	1.000	0.833	0.667	0.417	0.333	0.250
	activation	ReLU, LayerNorm	6	1.000	1.000	1.000	0.833	0.750	0.500
	output	residual (Δx)	18	1.000	1.000	1.000	0.917	0.917	0.833
	parameters	118,355	36	1.000	1.000	1.000	1.000	1.000	0.833
Forward training	optimiser	Adam [8]							
	learning rate	1×10^{-3}							
	weight decay	0							
	batch size	256							
	epochs	200 (early stop patience 20)							
	val split	every 5th source episode							
Multi-step loss	K values tested	1, 4, 8							
Stage A g_ϕ	input dim	8 (3 ctrl + 4 sigma + 1 delay)							
	hidden layers	2×32							
	output	sigmoid scalar in $[0, 1]$							
	parameters	1,409							
Stage A training	optimiser	Adam							
	learning rate	1×10^{-3}							
	weight decay	1×10^{-3}							
	epochs	500							
Joint-PD controller	K_p, K_d	30, 3							
	action scale α	5							
	muscle mapping	$\text{clip}(\alpha \text{ReLU}(M^{34 \times 20} u_{\text{joint}}))$							
IK solver	method	damped Jacobian pseudoinverse	noise	delay	$K=0$	$K=0.25$	$K=0.5$	$K=0.75$	$K=1$
	max iter / tol	200 / 0.01 m	none	0	0.493	0.771	0.782	0.774	0.770
	damping λ^2	0.1	none	18	0.497	0.498	0.708	0.746	0.697
BC policy	state dim variants	83 (full) / 80 / 77	high	0	0.493	0.767	0.780	0.771	0.770
	hidden layers	2×256	high	18	0.497	0.538	0.696	0.746	0.691
	output	sigmoid 34-dim	xhigh	0	0.493	0.772	0.769	0.766	0.762
	demos / training	30 episodes $\times$ 600 steps, 100 ep	xhigh	18	0.497	0.550	0.713	0.749	0.700
Eval grid (open-loop)	noise levels	none, low, medium, high, vhigh, xhigh							
	delays (steps)	0, 6, 18, 36							
	K values	0.0, 0.25, 0.5, 0.75, 1.0							
Eval grid (closed-loop)	noise levels (focused)	none, high, xhigh							
	delays (focused)	0, 18							

TABLE II
FORWARD-MODEL ROLLOUT MSE AND TIP PREDICTION ERROR ON THE HELD-OUT VALIDATION EPISODES. LOWER IS BETTER. ALL THREE MODELS SHARE THE SAME ARCHITECTURE AND TRAINING SET (51,534 TRANSITIONS ACROSS 106 EPISODES); THEY DIFFER ONLY IN THE MULTI-STEP VALUE K .

Model	MSE			Tip err. (m)		
	$h=1$	$h=10$	$h=50$	$h=1$	$h=10$	$h=50$
$K=1$ (single-step)	0.0055	0.0124	0.0524	0.0076	0.0355	0.1381
$K=4$ multi-step	0.0053	0.0084	0.0164	0.0088	0.0247	0.0635
$K=8$ multi-step	0.0055	0.0082	0.0114	0.0098	0.0215	0.0483

TABLE III
ORACLE KALMAN GAIN K^* AS A FUNCTION OF DELAY AND NOISE UNDER THE $K=8$ MULTI-STEP-SUPERVISED FORWARD MODEL (MEAN OVER 3 CONTROLLERS; 72 CONDITIONS TOTAL). 27 OF 72 CELLS REQUIRE $K^* < 1$.

per cell). The K-sweep is the basis of the closed-loop oracle K_{cl}^* defined in Sec. III-F. $K_{cl}^*=0$ in every cell. Two regularities are worth noting. First, at zero delay the K-curve is sharp around $K=0$: moving from $K=0$ to $K=0.25$ jumps min-tip from 0.49 m to 0.77 m. The Stage A predictor's sigmoid asymptote ($K \approx 0.025$) sits on the wrong side of this jump in delay-0 cells, which is why the residual gap remains (~ 17 cm; see Sec. III-F). Second, at delay-18 the K-curve is smooth near $K=0$ ($K=0$ and $K=0.25$ within 0.01–0.05 m) so the predictor's small offset is harmless and the same predictor matches the $K=0$ baseline within 1.5 cm.

TABLE IV
CLOSED-LOOP min-TIP ERROR (M) PER CELL ACROSS THE K -SWEEP UNDER THE $K=8$ FORWARD MODEL AND THE JOINT-PD CONTROLLER. EACH CELL AVERAGES 10 EPISODES. BOLD MARKS THE PER-CELL MINIMUM.

APPENDIX E

SOFTWARE, HARDWARE, AND REPRODUCIBILITY

All experiments run on a single workstation (no GPU is required; the forward model and Stage A predictor are small enough that CPU training takes only minutes). Software stack: Python 3.11, PyTorch (CPU), MuJoCo 2.3, MyoSuite 2.12.2, Gymnasium 0.29. Reproduction scripts and exact dataset paths are recorded in the source repository's docs/02_InitialImplementationPlan.md and docs/03_PaperOutline.md.

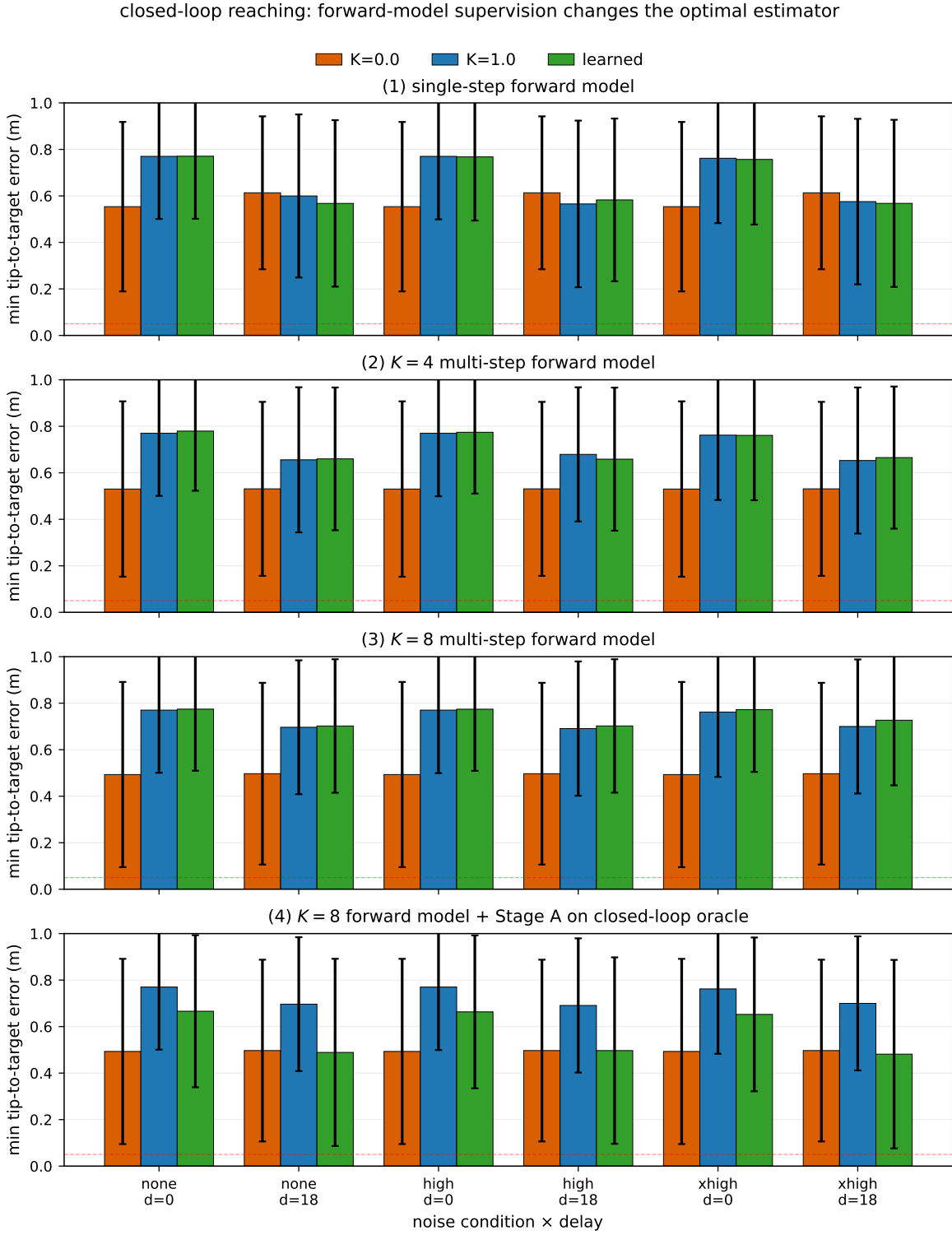


Fig. 6. Closed-loop reaching min-tip error under the joint-PD controller. Four forward-model / oracle variants. Top: single-step-supervised forward model. Second: $K=4$ multi-step. Third: $K=8$ multi-step. Bottom: $K=8$ forward model with Stage A retrained on the closed-loop oracle (Phase C). The orange $K=0$ bar drops monotonically as multi-step supervision strengthens; in panel 4 the green learned bar matches the $K=0$ baseline at delay-18 cells.